



Universidad Nacional Experimental de Guayana.
Vicerrectorado Académico
Proyecto de carrera: Ingeniería en informática.
Asignatura: Lenguajes y Compiladores.

Análisis Sintáctico

Profesor:
Félix Marquez

Alumnos:
S1. Palma Franyibeth CI 31.564.549
S1. Rodriguez Edfrank CI 30.857.264
S2. Jesus Baez CI 26.753.871
S2. Javier Useche CI 28.700.959

Ciudad Guayana, julio 2026.

Introducción

Dentro de las fases del compilador, el análisis sintáctico (parser) recibe la secuencia de tokens producida por el analizador léxico y valida que cumpla con las reglas de una gramática libre de contexto (GLC). Como resultado de este proceso, el parser no solo determina si la entrada es válida, sino que construye una representación intermedia que las siguientes fases (análisis semántico, optimización y generación de código) utilizarán: el Árbol de Sintaxis Abstracta (AST). Este informe desarrolla la Actividad 1 del Tema 5: se define el AST, se explican sus características principales, se presentan dos ejemplos de entrada y se implementa, en Python, un lexer y un parser recursivo descendente capaz de construir el AST correspondiente a cada ejemplo.

1. Árbol de Sintaxis Abstracta (AST)

Un Árbol de Sintaxis Abstracta (Abstract Syntax Tree, AST) es una estructura de datos jerárquica y en forma de árbol que representa la estructura sintáctica de un programa fuente, conservando únicamente los elementos que tienen significado semántico y descartando los detalles puramente sintácticos de la gramática concreta (paréntesis de agrupación, punto y coma, palabras reservadas usadas solo como delimitadores, no-terminales intermedios introducidos para resolver la precedencia o eliminar recursión izquierda, etc.). Cada nodo interno del AST representa un operador, una construcción del lenguaje (una asignación, una sentencia if, un bucle, la declaración de una función) o un constructor sintáctico, mientras que cada nodo hoja representa un operando: un identificador, una constante numérica, una cadena, etc. El AST es la salida natural del analizador sintáctico y la entrada natural del analizador semántico.

1.1 Características principales

- Abstracción: omite información redundante de la derivación (paréntesis, delimitadores, no-terminales auxiliares como E' o T' usados para eliminar recursión izquierda); solo se conserva lo necesario para reconstruir el significado del programa.
- Jerarquía y precedencia implícita: la estructura del árbol codifica por sí sola la precedencia y asociatividad de los operadores; no se requieren paréntesis para saber qué subexpresión se evalúa primero (la subexpresión más profunda se evalúa antes).
- Nodos tipados: cada nodo corresponde a una construcción semántica concreta del lenguaje (BinOp, Assign, If, Ident, Num, etc.), lo que facilita recorridos (visitor pattern) durante el análisis semántico y la generación de código.
- Compacidad: en comparación con el árbol de análisis o parse tree (que refleja cada paso de la derivación de la gramática), el AST es considerablemente más pequeño y legible.
- Independencia de la gramática concreta: distintas gramáticas (o distintas formas de escribir la misma gramática, p. ej. eliminando recursión izquierda) pueden producir el mismo AST para una misma entrada.
- Base para fases posteriores: sirve de entrada al chequeo de tipos, la resolución de ámbitos, la optimización (eliminación de nodos redundantes, plegado de constantes) y la generación de código intermedio u objeto.

1.2 AST vs. árbol de análisis (parse tree)

Es importante distinguir el AST del árbol de análisis o parse tree. El parse tree (o árbol de derivación) refleja fielmente cada producción de la gramática aplicada durante la derivación, incluyendo terminales sintácticos sin significado semántico propio (paréntesis, comas, punto y coma) y no-terminales auxiliares. El AST, en cambio, es una versión depurada y abstracta de ese árbol: se conservan los operadores y operandos, pero se eliminan los símbolos que solo sirven para guiar el análisis. Por ejemplo, para la expresión $(2 + 3)$, el parse

tree contendría nodos para “(” y “)”, mientras que el AST solo contendría el nodo BinOp(‘+’) con sus dos operandos.

Aspecto	Árbol de análisis (Parse Tree)	Árbol de Sintaxis Abstracta (AST)
Nivel de detalle	Refleja cada producción de la gramática	Solo retiene información semánticamente relevante
Tamaño	Grande (incluye no-terminales y delimitadores)	Compacto
Uso típico	Depuración del propio parser	Entrada para análisis semántico, optimización y generación de código
Dependencia de la gramática	Alta (cambia si la gramática se reescribe)	Baja (varias gramáticas equivalentes generan el mismo AST)

1.3 Ejemplos de entrada y construcción del AST

A continuación, se presentan dos ejemplos de entrada y el AST correspondiente. Para cada uno se muestra la gramática (GLC) utilizada, la entrada concreta, el AST resultante (representación textual y diagrama) y un fragmento del código Python que lo construye. La implementación completa se incluye en la Sección 4 y en el archivo anexo ast_ejemplos.py.

Ejemplo 1 — Expresión aritmética

Se utiliza la gramática clásica de expresiones aritméticas con suma, resta, multiplicación, división y agrupación mediante paréntesis, reescrita para eliminar la recursión izquierda y permitir un análisis descendente LL(1):

```

E -> T E'
E' -> + T E' | - T E' | epsilon
T -> F T'
T' -> * F T' | / F T' | epsilon
F -> ( E ) | NUM

```

Entrada de prueba:

3 + 4 * (2 - 1)

Al aplicar la precedencia habitual (la multiplicación y la división se agrupan antes que la suma y la resta, y el paréntesis fuerza la evaluación de “2 - 1” primero), el AST resultante es el siguiente:

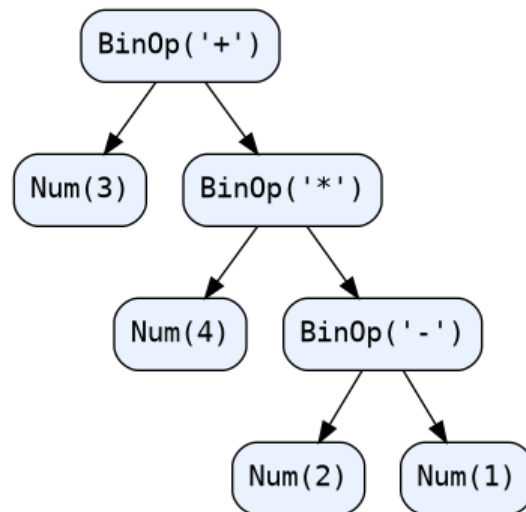


Figura 1. AST de la expresión $3 + 4 * (2 - 1)$

Nótese que el árbol no contiene nodos para los paréntesis: la jerarquía por sí sola (el nodo `BinOp('*')` como hijo derecho de la raíz, y `BinOp('-')` como hijo derecho de este) ya codifica que “ $4 * (2 - 1)$ ” se evalúa como una unidad y que “ $2 - 1$ ” se evalúa antes que la multiplicación.

Ejemplo 2 — Sentencia condicional (if / else)

El segundo ejemplo corresponde a una sentencia condicional de un mini-lenguaje tipo C, con una condición relacional y una asignación en cada rama:

```

IfStmt -> if ( Expr OP Expr ) { Assign } else { Assign }
Assign -> ID = Expr ;
Expr   -> Atom + Expr | Atom
Atom   -> ID | NUM
OP      -> > | < | == | >= | <=
  
```

Entrada de prueba:

```
if (x > 3) { y = x + 1; } else { y = 0; }
```

El AST agrupa la construcción en tres partes claramente diferenciadas —condición, rama verdadera y rama falsa— y ya no conserva los símbolos “if”, “(”, “)”, “{”, “}” ni “;”, que solo cumplieron su función durante el análisis sintáctico:

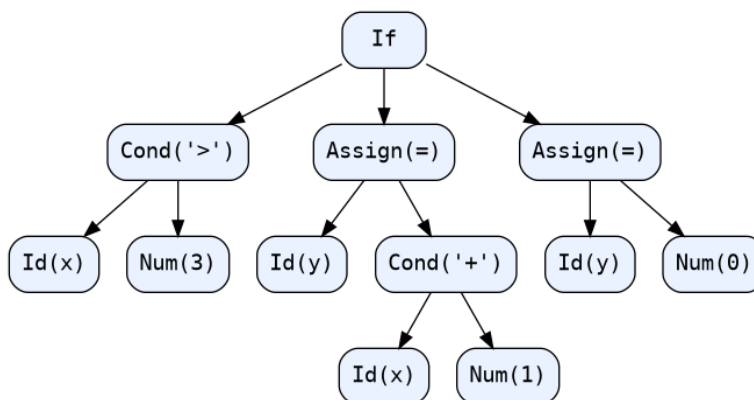


Figura 2. AST de la sentencia `if (x > 3) { y = x + 1; } else { y = 0; }`

1.4 Implementación

La implementación se realizó en Python 3 y consta de tres partes: (1) una clase base `ASTNode` común a ambos ejemplos, con métodos para imprimir el árbol en forma textual y para exportarlo como diagrama mediante la librería `Graphviz`; (2) un lexer basado en expresiones regulares y un parser recursivo descendente (LL(1)) para expresiones aritméticas (Ejemplo 1); y (3) un lexer y un parser recursivo descendente para la sentencia `if/else` (Ejemplo 2). Ambos parsers siguen el mismo principio: cada método del parser corresponde a un no-terminal de la gramática y, al reconocer una producción, construye y devuelve el nodo de AST correspondiente en lugar de un nodo de árbol de análisis genérico.

Clase base del nodo de AST

class `ASTNode`:

```

def children(self) -> List["ASTNode"]:
    return []

def label(self) -> str:
    return self.__class__.__name__

def pretty(self, prefix="", is_last=True, is_root=True) -> str:
    # imprime el arbol con conectores tipo "|--" y "--"
    ...

```

Parser recursivo descendente — Ejemplo 1 (expresiones)

El núcleo del parser recorre la gramática sin recursión izquierda: `parse_E` maneja `+` y `-`, `parse_T` maneja `*` y `/`, y `parse_F` resuelve un número o una subexpresión entre paréntesis. Cada función retorna directamente un nodo `BinOp` o `Num`:

```

def parse_E(self) -> ASTNode:
    node = self.parse_T()
    while self.peek()[0] == "OP" and self.peek()[1] in ("+", "-"):
        op = self.advance()[1]
        right = self.parse_T()
        node = BinOp(op, node, right)    # construccion del nodo AST

```

```

return node

def parse_F(self) -> ASTNode:
    tok = self.peak()
    if tok[0] == "NUM":
        self.advance()
        return Num(float(tok[1]) if "." in tok[1] else int(tok[1]))
    if tok[0] == "LPAREN":
        self.advance()
        node = self.parse_E()
        self.expect("RPAREN")      # el parentesis se consume
        return node                # pero NO se agrega al AST

```

Parser recursivo descendente — Ejemplo 2 (if / else)

De forma análoga, `parse_if` reconoce la condición y las dos ramas, construyendo un único nodo `IfElse`; los símbolos de puntuación (paréntesis, llaves, punto y coma) se consumen con `expect()` pero nunca se incorporan al árbol:

```

def parse_if(self) -> IfElse:
    self.expect("IF", "if")
    self.expect("LPAREN")
    left = self.parse_expr()
    op = self.expect("OP")[1]
    right = self.parse_expr()
    cond = Condition(op, left, right)
    self.expect("RPAREN")
    self.expect("LBRACE")
    then_branch = self.parse_assign()
    self.expect("RBRACE")
    self.expect("ELSE", "else")
    self.expect("LBRACE")
    else_branch = self.parse_assign()
    self.expect("RBRACE")
    return IfElse(cond, then_branch, else_branch)

```

1.5 Ejecución y salida del programa

Al ejecutar `python3 ast_ejemplos.py` se procesan ambos ejemplos y se imprime el AST en forma de árbol textual (además de generarse el diagrama gráfico mostrado en la Sección 3). La salida obtenida fue la siguiente:

```

Entrada : 3 + 4 * (2 - 1)
AST (forma de arbol):
BinOp('+')
|-- Num(3)
`-- BinOp('*')
    |-- Num(4)
    `-- BinOp('-')
        |-- Num(2)
        `-- Num(1)

```

Resultado de evaluar el AST -> 7

Entrada : if (x > 3) { y = x + 1; } else { y = 0; }

AST (forma de arbol):

```
If
|-- Cond('>')
|   |-- Id(x)
|   `-- Num(3)
|-- Assign(=)
|   |-- Id(y)
|   `-- Cond('+')
|       |-- Id(x)
|       `-- Num(1)
`-- Assign(=)
    |-- Id(y)
    `-- Num(0)
```

Actividad 2: COMPARACION ENTRE ANALISIS LL Y LR

2.1 Definiciones Fundamentales

Parser LL (Left-to-right, Leftmost derivation)

Los parsers LL realizan un analisis **descendente** (top-down), construyendo el arbol de derivacion desde la raiz (simbolo inicial) hacia las hojas (tokens). La denominacion indica:

Left-to-right: Lee la entrada de izquierda a derecha

Leftmost derivation: Realiza la derivacion por la izquierda

El parser LL mas comun es el **recursivo descendente**, donde cada no-terminal de la gramatica se implementa como una funcion que reconoce las producciones correspondientes. La variante **LL(1)** utiliza un solo token de lookahead para decidir que produccion aplicar.

Parser LR (Left-to-right, Rightmost derivation)

Los parsers LR realizan un analisis **ascendente** (bottom-up), construyendo el arbol desde las hojas hacia la raiz mediante **reducciones**. La denominacion indica:

Left-to-right: Lee la entrada de izquierda a derecha

Rightmost derivation: Construye la derivacion por la derecha (en sentido inverso)

Los parsers LR son **tabla-driven**: utilizan una pila de estados y una tabla de analisis (ACTION/GOTO) para decidir entre operaciones de **shift** (desplazar token a la pila) o **reduce** (reducir produccion).

2.2 Tabla Comparativa Detallada

Criterio	LL (Recursivo Descendente)	LR (Tabla-Driven)
Estrategia	Descendente (Top-Down)	Ascendente (Bottom-Up)
Derivacion	Por la izquierda	Por la derecha (inversa)
Construccion del arbol	De raiz a hojas	De hojas a raiz
Complejidad de implementacion	Baja (funciones recursivas)	Alta (tablas grandes)
Complejidad algoritmica	$O(n)$	$O(n) - O(n^4)$ teorico para ALL(*)
Memoria requerida	Pila de llamadas ($O(\text{depth})$)	Pila de estados + tabla ($O(n)$)
Gramaticas soportadas	No ambiguas, sin recursion izquierda	Gramaticas mas generales, incluye recursion izquierda
Deteccion de errores	Inmediata (al primer token inesperado)	Retardada (hasta que no puede reducir)
Mensajes de error	Faciles de personalizar	Mas cripticos (dependen de la tabla)
Mantenimiento	Facil de modificar manualmente	Requiere regenerar tablas
Rendimiento	Rapido (llamadas a funciones)	Muy rapido (acceso a tabla)
Herramientas	ANTLR (ALL(*)), JavaCC, Coco/R	Bison/Yacc (LALR), Menhir (LR(1)), Happy

2.3 Evidencia de Uso Actual en la Industria (2025-2026)

Uso de LL Parsers

Segun investigaciones recientes, los parsers LL dominan en el desarrollo moderno de lenguajes y herramientas de productividad:

ANTLR 4 (Adaptive LL(*)): Es el parser generator mas utilizado en la actualidad. Se emplea en:

Hibernate (ORM de Java)

Twitter Search (query parsing)

Hadoop (Hive, Pig)

Oracle SQL Developer IDE

Presto/Trino (motor SQL distribuido)

Spark SQL (Apache Spark)

Rust: El compilador de Rust utiliza un parser recursivo descendente hand-written, aprovechando las ventajas de LL para mensajes de error excepcionales.

Go: El compilador oficial (gc) usa un parser recursivo descendente LL(1) manual.

TypeScript: El parser de Microsoft es un recursivo descendente manual que prioriza la recuperacion de errores y la resiliencia.

Python: A partir de Python 3.9, el parser oficial migro de LL(1) a PEG (Parsing Expression Grammar), que es una extension de LL con lookahead ilimitado.

Uso de LR Parsers

Los parsers LR mantienen su relevancia en sistemas tradicionales y academicos:

GCC: El compilador GNU C utiliza un parser LALR(1) generado con Bison.

PostgreSQL: El parser SQL es generado con Bison (LALR).

MySQL/MariaDB: Utilizan Bison para el parsing SQL.

PHP: El parser oficial se genera con Bison.

Ruby: El parser original (YARV) era generado con Bison, aunque versiones recientes han migrado a parsers hand-written.

Universidades: Segun un estudio empirico de 2021 sobre las top 16 universidades del mundo (MIT, Stanford, Berkeley, ETH Zurich, etc.), 13 de 16 utilizan variantes de Lex/Yacc/Bison (LR) en sus cursos de compiladores, mientras que solo MIT usa ANTLR (LL) como herramienta principal.

Tendencias Actuales

Migracion a LL: Muchos lenguajes modernos (Rust, Go, TypeScript, Swift) prefieren parsers recursivos descendentes hand-written por la calidad de errores y facilidad de depuracion.

Herramientas hibridas: ANTLR 4 con su algoritmo ALL(*) ha cerrado la brecha de poder expresivo, manejando gramaticas que tradicionalmente requerian LR.

Parser Combinators: En lenguajes funcionales (Haskell, Scala, Rust), los parser combinators (variantes de LL) ganan popularidad por su composabilidad.

2.4 Implementaciones para el Mismo Lenguaje

Se han implementado dos parsers completos para una **Calculadora de Expresiones Aritmeticas** que soporta:

Numeros enteros

Operadores: +, -, *, /

Parentesis para agrupacion

Precedencia: * y / tienen mayor precedencia que + y -

Asociatividad: Izquierda

Implementacion LL(1): Parser Recursivo Descendente

Gramatica LL(1) utilizada (sin recursion izquierda, factorizada):
$$E \rightarrow T E1$$

$$E1 \rightarrow + T E1 \mid - T E1 \mid \text{epsilon}$$

$$T \rightarrow F T1$$

$$T1 \rightarrow * F T1 \mid / F T1 \mid \text{epsilon}$$

$$F \rightarrow (E) \mid \text{NUM}$$
Características de la implementacion:

- Lexer basado en AFD (Automata Finito Determinista)
- Parser con funciones recursivas por cada no-terminal
- Construccion de AST (Arbol de Sintaxis Abstracta)
- Evaluacion del AST mediante recorrido recursivo
- Recuperacion basica de errores (avance al siguiente token)
- **Archivo:** implementaciones/ll_parser/parser_ll.py
- Implementacion LR(1): Parser Tabla-Driven

Gramatica LR(1) utilizada (con recursion izquierda permitida):
$$0. S1 \rightarrow S$$

$$1. S \rightarrow E$$

$$2. E \rightarrow E + T$$

$$3. E \rightarrow E - T$$

$$4. E \rightarrow T$$

$$5. T \rightarrow T * F$$

$$6. T \rightarrow T / F$$

$$7. T \rightarrow F$$

$$8. F \rightarrow (E)$$

$$9. F \rightarrow \text{NUM}$$

Características de la implementación:

- Lexer idéntico al LL para comparación justa
- Tabla ACTION/GOTO construida manualmente (16 estados)
- Algoritmo de análisis bottom-up con pila de estados
- Construcción de AST durante las reducciones
- Evaluación bottom-up (los resultados se propagan desde las hojas)
- Manejo de errores con detección de tokens inesperados
- **Archivo:** implementaciones/lr_parser/parser_lr.py

Resultados de Prueba Comparativa

Expresión de Entrada	Resultado LL(1)	Resultado LR(1)	Pasos LR
$3 + 5$	8	8	10
$10 - 2 * 3$	4	4	14
$(4 + 6) / 2$	5.0	5.0	19
$8 / 2 + 3 * 2$	10.0	10.0	18
42	42	42	5
$(1 + 2) * (3 + 4)$	21	21	29

Análisis de resultados:

Ambos parsers producen **resultados idénticos**, validando la equivalencia semántica

El parser LR requiere más pasos de análisis (operaciones shift/reduce) debido a la naturaleza bottom-up

El parser LL construye el árbol de forma más directa (top-down), mientras que LR requiere reducciones intermedias

El AST de LR refleja la estructura bottom-up con nodos intermedios de reducción visibles

Actividad 3: GENERADORES DE ANALIZADORES SINTACTICOS Y GESTION DE ERRORES**3.1 Resumen de Generadores de Analizadores Sintacticos****3.1.1 ANTLR (Another Tool for Language Recognition)**

Descripción: ANTLR es el parser generator más popular y moderno, desarrollado por Terence Parr. La versión 4 utiliza el algoritmo **ALL(*)** (Adaptive LL(*)), que combina la simplicidad de LL con el poder expresivo cercano a LR.

Características:

- Soporta multiples lenguajes objetivo: Java, C#, C++, Python, JavaScript, Swift, Go, PHP
- Gramatica en formato EBNF (mas legible que BNF)
- Generacion automatica de arboles de parseo (parse trees)
- Separacion entre gramatica y codigo de acciones semanticas (patrones Visitor/Listener)
- IDE propio (ANTLRWorks) para visualizacion y debugging de gramaticas
- Soporte completo para Unicode
- Plugin para IntelliJ, NetBeans, Eclipse, Visual Studio

Uso en la industria: Twitter, Hadoop, Hive, Presto, Oracle SQL Developer, Hibernate.

Gestion de errores:

Recuperacion por modo panico (panic mode): ANTLR descarta tokens hasta encontrar uno de sincronizacion

Recuperacion por reglas de seguimiento: Utiliza tokens FOLLOW para decidir cuando detener la recuperacion

Mensajes de error contextuales: Reporta la regla gramatical donde ocurrio el error

Recuperacion en expresiones: Maneja especialmente bien errores en expresiones anidadas

3.1.2 Bison (GNU Parser Generator)

Descripcion: Bison es la implementacion GNU de Yacc (Yet Another Compiler Compiler). Genera parsers LALR(1), LR(1), IELR(1) y GLR.

Caracteristicas:

- Lenguajes de salida: C, C++, D, Java, XML
- Gramatica en formato BNF/Yacc
- Requiere lexer externo (generalmente Flex)
- Licencia GPL con excepcion para parsers generados
- Soporta recursion izquierda (directa e indirecta)
- GLR (Generalized LR) para gramaticas ambiguas

Uso en la industria: GCC, PostgreSQL, MySQL, PHP, BASH.

Gestion de errores:

Token error: Permite definir reglas de recuperacion en la gramatica misma

Recuperacion por insercion/eliminacion: Bison puede insertar tokens faltantes o eliminar tokens sobrantes

Mensajes de error crípticos: Heredado de Yacc, los mensajes suelen referirse a estados de la tabla LR

Depuracion con trazas: Opcion --debug para seguir las acciones shift/reduce

3.1.3 Flex (Fast Lexical Analyzer Generator)

Descripcion: Flex genera analizadores lexicos (lexers) basados en AFD. Aunque no es un parser generator, es inseparable de Bison/Yacc en la mayoria de implementaciones.

Características:

Expresiones regulares para definir tokens

Genera codigo C/C++

Estados/modos lexicos para contextos

Integracion directa con Bison mediante token codes

Gestion de errores:

Regla <<EOF>> para manejar fin de archivo

Macro ECHO para caracteres no reconocidos

Posibilidad de definir reglas catch-all para errores lexicos

3.1.4 JavaCC (Java Compiler Compiler)

Descripcion: Generador de parsers LL(k) para Java. Similar a ANTLR pero mas antiguo y con menos características modernas.

Características:

- Parser y lexer integrados en una sola especificacion
- Lookahead de k tokens (LL(k))
- Genera codigo Java puro
- Soporte para acciones semanticas en Java

Gestion de errores:

- Excepciones ParseException con informacion de linea/columna

- Recuperacion por modo panico basico
- Mensajes de error detallados incluyendo token esperado vs encontrado

3.1.5 Menhir

Descripción: Generador de parsers LR(1) para OCaml. Es la evolucion de OCamlYacc con mejor manejo de conflictos y mensajes de error.

Características:

- Algoritmo LR(1) puro (mas potente que LALR)
- Conflictos resueltos con precedencias explicitas
- Codigo OCaml tipado estaticamente
- Usado por el compilador de OCaml y Coq

Gestion de errores:

- Sistema de mensajes de error personalizables
- Deteccion de conflictos shift/reduce en tiempo de generacion
- Recuperacion mediante puntos de recuperacion explicitos

3.1.6 PLY (Python Lex-Yacc)

Descripcion: Implementacion de Lex/Yacc puramente en Python. Usa introspeccion y reflexion para definir reglas como docstrings.

Características:

- LALR(1) parser generator
- Puro Python, sin dependencias externas
- Facil de usar para prototipos rapidos
- Usado en proyectos academicos y pequenos DSLs

Gestion de errores:

- Funcion `p_error()` personalizable
- Recuperacion por modo panico
- Mensajes de error con pila de simbolos

3.2 Comparativa de Gestion de Errores

Generador	Tipo Parser	Estrategia Principal	Calidad de Mensajes	Recuperacion Avanzada
ANTLR 4	ALL (*)	Modo panico + tokens FOLLOW	Excelente (contextual)	Si (recuperacion inteligente en expresiones)
Bison	LALR/LR/GLR	Token error en gramatica	Regular (criptica)	Si (insercion/eliminacion de tokens)
JavaCC	LL(k)	Excepciones ParseException	Buena	Basica (modo panico)
Menhir	LR(1)	Puntos de recuperacion	Muy buena (personalizable)	Si (mediante %on_error_reduce)
PLY	LALR(1)	Funcion p_error()	Regular	Basica

3.3 Tecnicas Comunes de Recuperacion de Errores

Modo Panico (Panic Mode): Descartar tokens hasta encontrar uno de sincronizacion (generalmente ;, }, o fin de linea). Usado por ANTLR y la mayoría de generadores.

Recuperacion por Frase (Phrase Recovery): Saltar una cantidad fija de tokens y continuar. Menos agresivo que el modo panico.

Recuperacion por Reglas de Error: Definir producciones explicitas con el token especial error (caracteristico de Bison/Yacc).

Correccion Local: Insertar, eliminar o sustituir un solo token para continuar el analisis. Requiere calculo de costos.

Recuperacion Semantica: Utilizar informacion de tipos o contexto para sugerir correcciones (usado en IDEs modernos con analisis semantico integrado).

Reporte de Múltiples Errores: Continuar el parsing despues del primer error para encontrar errores subsiguientes. ANTLR y Bison soportan esto nativamente.

Conclusiones

El AST es la estructura central que conecta el análisis sintáctico con el resto del pipeline del compilador: es compacto, tipado y libre de ruido sintáctico, lo que lo hace apto para chequeo de tipos, optimización y generación de código.

Un mismo AST puede construirse a partir de gramáticas escritas de forma distinta (por ejemplo, con o sin eliminación de recursión izquierda), siempre que ambas reconozcan el mismo lenguaje; el AST abstrae esas diferencias de la gramática concreta.

Un parser recursivo descendente LL(1) es una forma directa y didáctica de construir un AST: basta con que cada función que reconoce un no-terminal retorne el nodo correspondiente en lugar de únicamente validar la entrada.

Los símbolos puramente delimitadores (paréntesis, llaves, punto y coma, la propia palabra reservada “if”) se consumen durante el análisis pero deliberadamente no se agregan al árbol, lo cual es precisamente lo que distingue a un AST de un árbol de análisis (parse tree) completo.

REFERENCIAS BIBLIOGRAFICAS

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson Addison Wesley. (Libro "Dragon")
2. Parr, T. (2013). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf.
3. Levine, J. R., Mason, T., & Brown, D. (1992). *Lex & Yacc* (2nd ed.). O'Reilly Media.
4. Tomassetti, F. (2020). "Why you should not use (f)lex, yacc and bison". Strumenta. Recuperado de: <https://tomassetti.me/why-you-should-not-use-flex-yacc-and-bison/>
5. Gomez-Martinez, B., & Sierra-Rodriguez, J. L. (2021). "An empirical evaluation of Lex/Yacc and ANTLR parser generation tools in computer science education". *PMC/NIH*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC8893623/>
6. Parr, T., & Fisher, K. (2011). "LL(*): The Foundation of the ANTLR Parser Generator". *ACM SIGPLAN Notices*.
7. Wikipedia. "Comparison of parser generators". https://en.wikipedia.org/wiki/Comparison_of_parser_generators
8. GitHub Gist. "Per criterion comparison of Parser Generators and Parser Combinators" (2025). <https://gist.github.com/chshersh/27844477752359735dfa41ac184d3bf2>
9. Grune, D., & Jacobs, C. J. (2008). *Parsing Techniques: A Practical Guide* (2nd ed.). Springer.
10. Stack Overflow. "Advantages of Antlr (versus say, lex/yacc/bison)". <https://stackoverflow.com/questions/212900/advantages-of-antlr-versus-say-lex-yacc-bison>

Anexo A. Código fuente completo (ast_ejemplos.py)

Se adjunta a continuación el código fuente completo utilizado para generar ambos ejemplos (también disponible como archivo independiente ast_ejemplos.py).

```
"""
Actividad 1 - Tema 5: Analisis Sintactico
Arbol de Sintaxis Abstracta (AST): implementacion de dos ejemplos.

Ejemplo 1: Expresion aritmetica
Gramatica (GLC):
  E -> E + T | E - T | T
  T -> T * F | T / F | F
```



```

F -> ( E ) | NUM
(reescrita sin recursion izquierda para el parser recursivo descendente)
E -> T E'
E' -> + T E' | - T E' | epsilon
T -> F T'
T' -> * F T' | / F T' | epsilon
F -> ( E ) | NUM

```

Ejemplo 2: Sentencia condicional (if-else) de un mini-lenguaje tipo C

Gramatica (GLC):

```

Stmt -> IfStmt | Assign
IfStmt -> if ( Cond ) { Assign } else { Assign }
Assign -> ID = Expr ;
Cond -> Expr OP Expr
Expr -> ID | NUM | Expr + Expr
OP -> > | < | == | >= | <=

```

Autor: Actividad Lenguaje y Compiladores - UNEG

"""

```

from __future__ import annotations
import re
import sys
from dataclasses import dataclass, field
from typing import List, Optional

```

try:

```

    import graphviz
    HAS_GRAPHVIZ = True
except ImportError:
    HAS_GRAPHVIZ = False

```

```

# =====
# 1. DEFINICION GENERICA DE NODO DE AST
# =====
class ASTNode:
    """

```

```

    Clase base de todo nodo del Arbol de Sintaxis Abstracta.
    Un AST, a diferencia de un arbol de analisis (parse tree), NO
    conserva simbolos superfluos de la gramatica (parentesis, punto y
    coma, no-terminales intermedios); solo retiene la informacion
    semanticamente relevante para las siguientes fases del compilador.
    """

```

```

    def children(self) -> List["ASTNode"]:
        """Devuelve la lista de nodos hijos (para recorrido/impression)."""
        return []

```

```

    def label(self) -> str:
        """Etiqueta textual del nodo (se muestra en el arbol)."""
        return self.__class__.__name__

```

----- utilidades de presentacion -----

```

def pretty(self, prefix: str = "", is_last: bool = True, is_root: bool = True) -> str:
    """Devuelve una representacion textual en forma de arbol (estilo 'tree')."""
    lines = []
    connector = "" if is_root else ("`-- " if is_last else "|-- ")
    lines.append(f"{prefix}{connector}{self.label()}")
    new_prefix = prefix if is_root else prefix + ("    " if is_last else "|    ")
    kids = self.children()
    for i, child in enumerate(kids):
        lines.append(child.pretty(new_prefix, i == len(kids) - 1, False))
    return "\n".join(lines)

```

```

def to_graphviz(self, g: "graphviz.Digraph", counter: List[int]) -> str:
    """Construye recursivamente el grafo graphviz y devuelve el id del nodo."""
    node_id = f"n{counter[0]}"
    counter[0] += 1

```

```

        g.node(node_id, self.label(), shape="box", style="rounded, filled",
               fillcolor="#EAF2FF", fontname="Consolas")
    for child in self.children():
        child_id = child.to_graphviz(g, counter)
        g.edge(node_id, child_id)
    return node_id

def render(self, filename: str, fmt: str = "png"):
    if not HAS_GRAPHVIZ:
        raise RuntimeError("graphviz no disponible")
    g = graphviz.Digraph(filename, format=fmt)
    g.attr("graph", rankdir="TB", bgcolor="white")
    self.to_graphviz(g, [0])
    g.render(filename, cleanup=True)

# =====
# 2. NODOS ESPECIFICOS - EJEMPLO 1: EXPRESIONES ARITMETICAS
# =====
@dataclass
class Num(ASTNode):
    value: float

    def label(self) -> str:
        return f"Num({self.value})"

@dataclass
class BinOp(ASTNode):
    op: str
    left: ASTNode
    right: ASTNode

    def label(self) -> str:
        return f"BinOp('{self.op}')"

    def children(self) -> List[ASTNode]:
        return [self.left, self.right]

    def eval(self):
        l = self.left.eval() if isinstance(self.left, (BinOp, Num)) else None
        r = self.right.eval() if isinstance(self.right, (BinOp, Num)) else None
        return {"+": l + r, "-": l - r, "*": l * r, "/": l / r}[self.op]

# permitir eval() tambien sobre Num
def _num_eval(self):
    return self.value
Num.eval = _num_eval

# ----- Lexer para expresiones -----
TOKEN_SPEC_EXPR = [
    ("NUM", r"\d+(\.\d+)?"),
    ("OP", r"[+ \- * /]"),
    ("LPAREN", r"("),
    ("RPAREN", r")"),
    ("SKIP", r"[ \t]+"),
]
TOKEN_RE_EXPR = re.compile("|".join(f"(?P<{n}>{p})" for n, p in TOKEN_SPEC_EXPR))

def tokenize_expr(text: str):
    tokens = []
    for m in TOKEN_RE_EXPR.finditer(text):
        kind = m.lastgroup
        val = m.group()
        if kind == "SKIP":
            continue

```

```

        tokens.append((kind, val))
    tokens.append(("EOF", ""))
    return tokens

class ExprParser:
    """
    Parser recursivo descendente (LL(1)) para expresiones aritmeticas.
    Implementa la gramatica sin recursion izquierda:
        E -> T E'
        E' -> + T E' | - T E' | epsilon
        T -> F T'
        T' -> * F T' | / F T' | epsilon
        F -> ( E ) | NUM
    """

    def __init__(self, text: str):
        self.tokens = tokenize_expr(text)
        self.pos = 0

    def peek(self):
        return self.tokens[self.pos]

    def advance(self):
        tok = self.tokens[self.pos]
        self.pos += 1
        return tok

    def expect(self, kind):
        tok = self.peek()
        if tok[0] != kind:
            raise SyntaxError(f"Se esperaba {kind}, se encontro {tok}")
        return self.advance()

    def parse(self) -> ASTNode:
        node = self.parse_E()
        self.expect("EOF")
        return node

    def parse_E(self) -> ASTNode:
        node = self.parse_T()
        while self.peek()[0] == "OP" and self.peek()[1] in ("+", "-"):
            op = self.advance()[1]
            right = self.parse_T()
            node = BinOp(op, node, right)
        return node

    def parse_T(self) -> ASTNode:
        node = self.parse_F()
        while self.peek()[0] == "OP" and self.peek()[1] in ("*", "/"):
            op = self.advance()[1]
            right = self.parse_F()
            node = BinOp(op, node, right)
        return node

    def parse_F(self) -> ASTNode:
        tok = self.peek()
        if tok[0] == "NUM":
            self.advance()
            return Num(float(tok[1]) if "." in tok[1] else int(tok[1]))
        if tok[0] == "LPAREN":
            self.advance()
            node = self.parse_E()
            self.expect("RPAREN")
            return node
        raise SyntaxError(f"Token inesperado: {tok}")

# =====

```

```

# 3. NODOS ESPECIFICOS - EJEMPLO 2: SENTENCIA IF-ELSE
# =====
@dataclass
class Ident(ASTNode):
    name: str

    def label(self) -> str:
        return f"Id({self.name})"

@dataclass
class Literal(ASTNode):
    value: str

    def label(self) -> str:
        return f"Num({self.value})"

@dataclass
class Assign(ASTNode):
    target: Ident
    expr: ASTNode

    def label(self) -> str:
        return "Assign(=)"

    def children(self) -> List[ASTNode]:
        return [self.target, self.expr]

@dataclass
class Condition(ASTNode):
    op: str
    left: ASTNode
    right: ASTNode

    def label(self) -> str:
        return f"Cond('{self.op}')"

    def children(self) -> List[ASTNode]:
        return [self.left, self.right]

@dataclass
class IfElse(ASTNode):
    condition: Condition
    then_branch: ASTNode
    else_branch: ASTNode

    def label(self) -> str:
        return "If"

    def children(self) -> List[ASTNode]:
        return [self.condition, self.then_branch, self.else_branch]

# ----- Lexer para sentencia if -----
TOKEN_SPEC_STMT = [
    ("IF", r"\bif\b"),
    ("ELSE", r"\belse\b"),
    ("NUM", r"\d+(\.\d+)?"),
    ("ID", r"[A-Za-z_][A-Za-z0-9_]*"),
    ("OP", r"==|>|<|<=|>=|+|-|*|/|><"),
    ("ASSIGN", r"="),
    ("LPAREN", r"\("),
    ("RPAREN", r"\)"),
    ("LBRACE", r"\{"),
    ("RBRACE", r"\}"),
    ("SEMI", r";"),

```

```

    ("SKIP", r"[\t\n]+"),
]
TOKEN_RE_STMT = re.compile("|".join(f"(?P<{n}>{p})" for n, p in TOKEN_SPEC_STMT))

def tokenize_stmt(text: str):
    tokens = []
    for m in TOKEN_RE_STMT.finditer(text):
        kind = m.lastgroup
        val = m.group()
        if kind == "SKIP":
            continue
        tokens.append((kind, val))
    tokens.append(("EOF", ""))
    return tokens

class StmtParser:
    """
    Parser recursivo descendente para:
    IfStmt -> if ( Expr OP Expr ) { Assign } else { Assign }
    Assign -> ID = Expr ;
    Expr   -> ID | NUM | Expr + Expr   (asociativo izquierdo simple)
    """
    def __init__(self, text: str):
        self.tokens = tokenize_stmt(text)
        self.pos = 0

    def peek(self):
        return self.tokens[self.pos]

    def advance(self):
        tok = self.tokens[self.pos]
        self.pos += 1
        return tok

    def expect(self, kind, val=None):
        tok = self.peek()
        if tok[0] != kind or (val is not None and tok[1] != val):
            raise SyntaxError(f"Se esperaba {kind} {val or ''}, se encontro {tok}")
        return self.advance()

    def parse(self) -> ASTNode:
        node = self.parse_if()
        self.expect("EOF")
        return node

    def parse_if(self) -> IfElse:
        self.expect("IF", "if")
        self.expect("LPAREN")
        left = self.parse_expr()
        op = self.expect("OP")[1]
        right = self.parse_expr()
        cond = Condition(op, left, right)
        self.expect("RPAREN")
        self.expect("LBRACE")
        then_branch = self.parse_assign()
        self.expect("RBRACE")
        self.expect("ELSE", "else")
        self.expect("LBRACE")
        else_branch = self.parse_assign()
        self.expect("RBRACE")
        return IfElse(cond, then_branch, else_branch)

    def parse_assign(self) -> Assign:
        ident = self.expect("ID")[1]
        self.expect("ASSIGN")
        expr = self.parse_expr()

```

```

        self.expect("SEMI")
        return Assign(Ident(ident), expr)

def parse_expr(self) -> ASTNode:
    node = self.parse_atom()
    while self.peek()[0] == "OP" and self.peek()[1] == "+":
        self.advance()
        right = self.parse_atom()
        node = Condition("+", node, right) # reutilizamos nodo binario generico
    return node

def parse_atom(self) -> ASTNode:
    tok = self.peek()
    if tok[0] == "NUM":
        self.advance()
        return Literal(tok[1])
    if tok[0] == "ID":
        self.advance()
        return Ident(tok[1])
    raise SyntaxError(f"Token inesperado: {tok}")

# =====
# 4. PROGRAMA PRINCIPAL: construye y muestra ambos AST
# =====
def main():
    print("=" * 70)
    print("EJEMPLO 1: AST de una expresion aritmetica")
    print("=" * 70)
    entrada1 = "3 + 4 * (2 - 1)"
    print(f"Entrada : {entrada1}")
    ast1 = ExprParser(entrada1).parse()
    print("AST (forma de arbol):")
    print(ast1.pretty())
    print(f"Resultado de evaluar el AST -> {ast1.eval()}")
    if HAS_GRAPHVIZ:
        ast1.render("/home/claude/ast_activity/ast_ejemplo1")
        print("Diagrama guardado en ast_ejemplo1.png")

    print()
    print("=" * 70)
    print("EJEMPLO 2: AST de una sentencia if-else")
    print("=" * 70)
    entrada2 = "if (x > 3) { y = x + 1; } else { y = 0; }"
    print(f"Entrada : {entrada2}")
    ast2 = StmtParser(entrada2).parse()
    print("AST (forma de arbol):")
    print(ast2.pretty())
    if HAS_GRAPHVIZ:
        ast2.render("/home/claude/ast_activity/ast_ejemplo2")
        print("Diagrama guardado en ast_ejemplo2.png")

if __name__ == "__main__":
    main()

```